Aide-mémoire — Niveau intermédiaire

Imprimez-le (Ctrl + P) et gardez-le près du clavier

Packages de ce niveau

```
install.packages(c("readxl", "dplyr", "tidyr", "lubridate", "stringr", "ggplot2", "patchwork"))
# optional: "ggrepel" (labels that do not overlap), "sf" (maps, Advanced level)
```

I0 — Importer de vrais fichiers

> - Ouvrez d'abord le fichier (Excel / éditeur de texte) : où commence le tableau, quel séparateur, quels codes de valeurs manquantes ?
> - Ne modifiez jamais le fichier brut : toutes les corrections vont dans le script.

Fonction	Usage	Exemple
`readxl::excel_sheets()`	Noms des feuilles d'un classeur	`excel_sheets("data/seaweed_farming.xlsx")`
`read_excel(path, sheet, skip, range, na)`	Lire une feuille ; `skip` = lignes au-dessus du tableau	`read_excel(f, sheet = "lines", skip = 3)`
`read.csv(..., na.strings, check.names)`	CSV à virgule ; déclarer les codes manquants ; garder des noms de colonnes atypiques	`read.csv(f, na.strings = c("", "NA", "-999"))`
`read.csv2()`	CSV à `;` et virgule décimale (Excel français)	`read.csv2("data/x.csv")`
`as.data.frame()`	Transformer un tibble en data frame classique	`as.data.frame(lines)`
`dim()` , `str()` , `summary()`	Taille, structure, résumé de chaque colonne	`str(lines)`
`colSums(is.na(df))`	Valeurs manquantes par colonne	`colSums(is.na(farmers))`
`write.csv(df, file, row.names = FALSE)`	Enregistrer un checkpoint	`write.csv(lines, "data/clean.csv", row.names = FALSE)`

I1 — Dates

> - Une date doit être de classe `Date` pour être triée, soustraite et tracée correctement. Les dates Excel sont des nombres de jours depuis le 1899-12-30.

Fonction	Usage	Exemple
`as.Date(x, format)`	Texte → date (`%d` jour, `%m` mois, `%Y` année, `%b` nom du mois)	`as.Date("21/03/2025", format = "%d/%m/%Y")`
`as.Date(n, origin = "1899-12-30")`	Nombre Excel → date	`as.Date(45372, origin = "1899-12-30")`
`dmy()` , `ymd()` , `mdy()`	lubridate : lire des dates selon l'ordre de leurs parties	`dmy("21/03/2025")`
`year()` , `month()` , `wday()`	Extraire année, mois, jour de la semaine	`month(d)`
`format(d, "%Y-%m")`	Date → texte dans n'importe quel format	`format(d, "%b %Y")`
`d2 - d1` , `difftime()` , `as.numeric()`	Nombre de jours entre deux dates	`as.numeric(harvest - planting)`
`floor_date(d, "month")`	Arrondir au mois / à la semaine	`floor_date(d, "week")`
`cut(x, breaks, labels)`	Nombres → classes	`cut(age, c(0, 30, 50, Inf))`

I2 — Restructurer

> - **Données rangées** : une variable par colonne, une observation par ligne. ggplot et dplyr veulent des données longues.

Fonction	Usage	Exemple
`pivot_longer(df, cols, names_to, values_to)`	Large → long	`pivot_longer(wide, -station, names_to = "month", values_to = "temp")`
`pivot_wider(df, names_from, values_from)`	Long → large	`pivot_wider(long, names_from = year, values_from = temp)`
`separate(df, col, into, sep)`	Séparer une colonne en plusieurs	`separate(df, month, c("year", "m"), sep = "-")`
`unite(df, new, a, b, sep)`	Coller des colonnes	`unite(df, "id", station, date)`
`group_by() |> summarise()`	Résumé par groupe	`group_by(zone) |> summarise(m = mean(temp, na.rm = TRUE))`

I3 — Jointures

> - Vérifiez d'abord la **clé** : même nom, même type, même orthographe, unique dans la table de référence (`sum(duplicated(clé))`). Comparez `nrow()` avant et après.

Fonction	Garde	R de base
`left_join(x, y, by = "key")`	Toutes les lignes de x	`merge(x, y, all.x = TRUE)`
`inner_join(x, y, by)`	Lignes présentes dans les deux	`merge(x, y)`
`full_join(x, y, by)`	Toutes les lignes des deux	`merge(x, y, all = TRUE)`
`anti_join(x, y, by)`	Lignes de x **sans** correspondance (pour trouver les erreurs)	`x[!x$key %in% y$key, ]`
`semi_join(x, y, by)`	Lignes de x avec correspondance, colonnes de x seulement	`x[x$key %in% y$key, ]`
`by = c("zone", "year", "month")`	Joindre sur plusieurs clés	

I4 — Données d'enquête et texte

Fonction	Usage	Exemple
`duplicated()` , `distinct()`	Trouver / supprimer les doublons	`h[!duplicated(h), ]`
`str_squish()` , `str_trim()`	Supprimer les espaces en trop	`str_squish(" Agree ")`
`str_to_lower/upper/title/sentence()`	Changer la casse	`str_to_sentence("STRONGLY agree")`
`str_detect(x, pattern)`	TRUE si le motif est présent	`str_detect(activities, "seaweed")`
`str_count()` , `str_replace_all()` , `str_sub()` , `str_split()`	Compter, remplacer, découper, séparer du texte	`str_count(x, ";") + 1`
`separate_rows(df, col, sep)`	Réponses à choix multiples → une ligne par réponse	`separate_rows(h, activities, sep = ";")`
`factor(x, levels, ordered = TRUE)`	Catégories ordonnées (échelle de Likert)	`factor(x, levels = c("Disagree", "Neutral", "Agree"), ordered = TRUE)`
`prop.table(table(), margin)` , `addmargins()`	Pourcentages par ligne (1) ou colonne (2), totaux	`round(prop.table(t, 1) * 100, 1)`

I5 — Conditions et boucles

Fonction	Usage	Exemple
`if (cond) {...} else {...}`	Une décision (un seul TRUE/FALSE)	`if (n < 5) message("small sample")`
`ifelse(test, yes, no)`	Décision vectorisée sur une colonne	`ifelse(sgr > 3.5, "good", "low")`
`case_when(cond ~ value, TRUE ~ other)`	Plusieurs classes	`case_when(t < 3 ~ "clear", t < 6 ~ "moderate", TRUE ~ "turbid")`
`for (i in seq_along(x)) {...}`	Répéter pour chaque élément	`for (v in villages) {...}`
`vector("list", n)` , `numeric(n)`	Conteneur vide à remplir dans une boucle	`res <- numeric(10)`
`sapply(x, f)` , `lapply(x, f)`	Appliquer une fonction à chaque élément (résultat vecteur / liste)	`sapply(split(w$sal, w$station), cv)`
`do.call(rbind, list)`	Empiler une liste de data frames	`do.call(rbind, results)`
`list.files()` , `file.path()` , `dir.create()`	Fichiers et dossiers dans les boucles	`list.files("data", pattern = "csv$")`

I6 — Fonctions

```
sgr <- function(w0, w1, days) {            # name <- function(arguments)
  if (any(w0 <= 0)) stop("weights must be positive")   # check the inputs
  100 * (log(w1) - log(w0)) / days         # the last value is returned
}
se <- function(x) sd(x, na.rm = TRUE) / sqrt(sum(!is.na(x)))
cv <- function(x) 100 * sd(x, na.rm = TRUE) / mean(x, na.rm = TRUE)
geo_mean <- function(x) exp(mean(log(x), na.rm = TRUE))
```

- Écrivez une fonction dès que vous copiez-collez le même code trois fois. Testez-la sur un cas calculable à la main.
- Les arguments peuvent avoir des valeurs par défaut : `function(x, digits = 2)` . `return()` seulement pour sortir plus tôt ; `stop()` / `warning()` pour les erreurs.

I7–I8 — Maîtriser ggplot2

Fonction	Usage
`labs(x, y, title, subtitle, caption, colour)`	Tous les textes du graphique, avec unités
`scale_colour_manual(values)` , `scale_*_brewer()` , `scale_*_viridis_c/d()`	Choisir les couleurs (viridis : adapté aux daltoniens)
`scale_y_continuous(breaks, labels)` , `scale_y_log10()` , `scale_x_date(date_breaks, date_labels)`	Graduations et étiquettes des axes
`coord_cartesian(ylim)`	**Zoomer** sans retirer de données (`ylim()` retire !)
`facet_wrap(~ g, scales = "free_y")` , `facet_grid(r ~ c)`	Petits multiples
`theme_minimal()` , `theme(legend.position, axis.text = element_text())`	Aspect général
`geom_pointrange(aes(ymin, ymax), position = position_dodge(0.5))`	Moyennes ± IC côte à côte
`geom_line()` , `geom_smooth()`	Séries temporelles, tendance
`reorder(g, x, median)`	Ordonner des catégories selon une valeur
`annotate("text" / "segment" / "rect", ...)` , `geom_hline()`	Ajouter notes, lignes, zones
`p1 + p2` , `p1 / p2` , `(p1 | p2) / p3` , `plot_annotation(tag_levels = "A")`	patchwork : combiner des graphiques
`ggsave(file, plot, width, height, units = "cm", dpi = 300)`	Exporter en PNG / PDF

I9 — Distributions, transformations, p-values

Fonction	Usage
`geom_histogram()` , `geom_density()`	Regarder d'abord la forme
`qqnorm(x); qqline(x)` / `stat_qq() + stat_qq_line()`	QQ-plot : des données normales suivent la droite
`shapiro.test(x)`	Test de normalité (faible avec petit n, trop strict avec grand n)
`rnorm()` , `dnorm()` , `pnorm()` , `qnorm()`	Loi normale : tirage, densité, probabilité, quantile
`log(x)` , `log1p(x)` , `sqrt(x)`	Transformer des données asymétriques / avec zéros / des comptages
`exp(mean(log(x)))`	Moyenne géométrique (re-transformée)
`replicate(n, expr)` , `sample(x)`	Simulations, permutations

> - **p-value** = probabilité d'un résultat au moins aussi extrême **si H0 était vraie**. Pas la probabilité que H0 soit vraie. Non significatif ≠ pas d'effet ; significatif ≠ important. Rapportez toujours la taille d'effet et son IC.

I10 — Choisir un test

Question	Paramétrique	Non paramétrique / exact
2 moyennes, groupes indépendants	`t.test(y ~ g, data)` (Welch)	`wilcox.test(y ~ g, data)`
2 moyennes, appariées (mêmes unités 2 fois)	`t.test(after, before, paired = TRUE)`	`wilcox.test(after, before, paired = TRUE)`
Proportions (2 variables catégorielles)	`chisq.test(table(a, b))` , `prop.test()` — attendus ≥ 5	`fisher.test(table(a, b))`
Relation (2 variables numériques)	`cor.test(x, y)` (Pearson, linear)	`cor.test(x, y, method = "spearman")` (monotonic)

> - **Tracez d'abord.** Choisissez le test d'après la question, le type des variables et l'appariement, **avant** de regarder p.
> - `chisq.test(t)$expected` vérifie les effectifs ; `tt$estimate` , `tt$conf.int` , `tt$p.value` extraient les résultats.
> - Relation courbe (monte puis descend) : aucun coefficient de corrélation ne convient — décrivez-la par un graphique.
> - Rapportez : « X était plus élevé en A qu'en B (moyennes, différence, IC 95 % ; test, statistique, ddl, p ; n) ».

I11 — Cartes avec ggplot2

Fonction	Usage
`aes(x = lon, y = lat)`	Longitude en x, latitude en y (sud = négatif)
`coord_quickmap(xlim, ylim)`	Rapport d'aspect correct et zoom
`geom_polygon(data = coast, aes(lon, lat), fill)`	Terre ; les lignes doivent rester dans leur ordre
`geom_point(data = ..., aes(size, fill), shape = 21)`	Points avec leurs propres données par couche
`geom_text(aes(label), hjust, nudge_x)`	Étiquettes
`scale_size_area()` , `scale_fill_viridis_c()`	Surface ∝ quantité ; couleurs séquentielles pour l'intensité
`annotate("segment", ..., arrow = arrow())`	Flèche du nord ; 0,09° de latitude ≈ 10 km
`sf::st_as_sf(df, coords = c("lon", "lat"), crs = 4326)` + `geom_sf()`	Aperçu du vrai SIG (niveau avancé)